FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

2

RULE – BASED AGENT

Student Name	**Kelzo Marcel O. Lumabas**
Section:	**AN42**
Professor:	**Ms. Crisola G. Tan**
Date:	**October 5, 2026**

I. BACKGROUND INFORMATION

A Rule-Based Agent is a type of intelligent agent that operates by following predefined rules to decide its actions. In this lab, students will work with a vacuum cleaner agent that can clean rooms or move between rooms depending on their state (dirty or clean).

The goal is to understand how agents perceive the environment, act according to rules, and update the environment.

II. TECHNICAL ASSESSMENT

RULE – BASED AGENT

Objective

At the end of this laboratory activity, students are expected to:

1. Explain the concept of rule-based agents.
2. Implement a rule-based agent in Python.
3. Interact with an environment by perceiving and acting based on rules.
4. Observe and analyze the behavior of an agent in a controlled environment.

Materials Needed

- Computer with Python 3.x installed
- Text editor or IDE (e.g., VSCode, PyCharm)
- Optional: Jupyter Notebook for interactive testing

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA2_lastname.pdf**
   (Example: TA2_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Laboratory Task

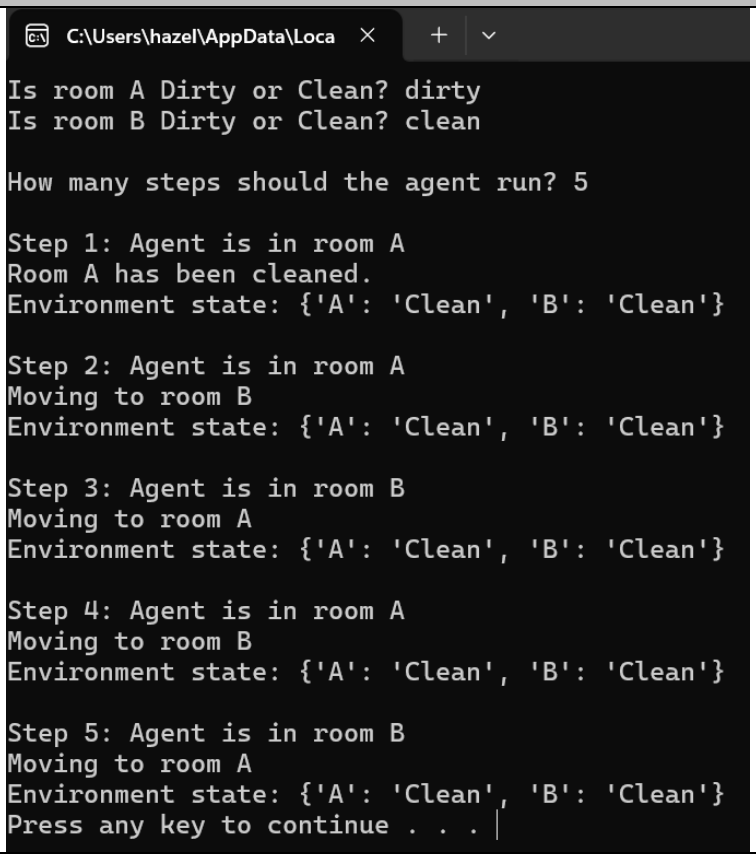
Pre-Lab Preparation

1. Review the Python concepts:
    - Classes and objects
    - If-Else statements
    - Loops (for loop)
    - Functions / methods
2. Understand the environment:
    - Two rooms: A and B
    - Each room can be Dirty or Clean

Task 1: Set Up the Environment

1. Initialize two rooms using a Python dictionary.
2. Allow the user to set the **initial state** of each room (dirty or clean).
3. Write a function to **check if a room is dirty** and a function to **clean a room**.

Sample Output



Task 2: Implement the Rule-Based Agent

1. Create a class for the agent.
2. Implement the **perceive-and-act method**:
    - Rule 1: If the current room is dirty, clean it.

- Rule 2: If the current room is clean, move to the other room.

3. Implement a **move method** to switch rooms.

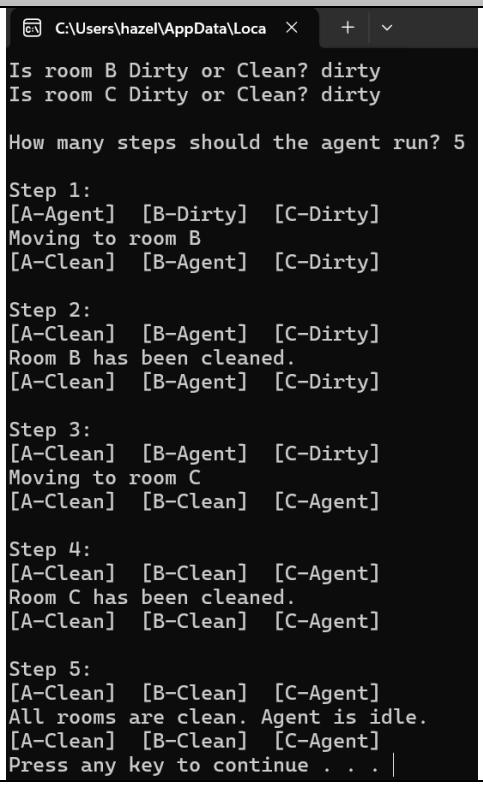
Task 3: Run the Simulation

1. Ask the user how many steps the simulation should run.
2. Run the agent in a loop for the specified steps.
3. After each step, print:
   - The agent's current location
   - The action taken (cleaned/moved)
   - Current state of the environment

Optional (Bonus Task)

- Add a **third room (C)** and update the agent logic.
- Make the agent **randomly choose which dirty room to clean next**.
- Visualize the agent moving between rooms using a **text-based grid**.

Sample Output (Bonus Task)



Your Code - Laboratory Tasks ⇩

```
# ========================================
# TASK 1: SET UP THE ENVIRONMENT
# ========================================

# Ask the user for the initial state of each room
room_a = input("Is room A Dirty or Clean? ").capitalize()
room_b = input("Is room B Dirty or Clean? ").capitalize()

# Store the rooms in a dictionary
environment = {
    "A": room_a,
    "B": room_b
}


# Function to check if a room is dirty
def check_dirty(room):
    return environment[room] == "Dirty"


# Function to clean a room
def clean_room(room):
    environment[room] = "Clean"


# ========================================
# TASK 2: RULE-BASED AGENT
# ========================================

class CleaningAgent:

    def __init__(self):
        # Agent starts in room A
        self.location = "A"

    # Move to the other room
    def move(self):
        if self.location == "A":
            self.location = "B"
```

```
        else:
            self.location = "A"

    # Perceive the current room and take action
    def perceive_and_act(self):

        # Rule 1:
        # If the current room is dirty, clean it
        if check_dirty(self.location):

            clean_room(self.location)

            return f"Room {self.location} has been cleaned."

        # Rule 2:
        # If the current room is clean, move
        else:

            self.move()

            return f"Moving to room {self.location}."


# ========================================
# TASK 3: RUN THE SIMULATION
# ========================================

# Ask the user how many steps to run
steps = int(input("\nHow many steps should the agent run? "))

# Create the agent
agent = CleaningAgent()


# Run the simulation
for step in range(1, steps + 1):

    print(f"\nStep {step}: Agent is in room {agent.location}")

    # Agent perceives the environment and acts
    action = agent.perceive_and_act()

    print(action)
```

```
    # Display current environment
    print("Environment state:", environment)
```

Task 4: Analysis Questions

After running the simulation, answer the following:

1. What rules does the agent follow?
2. How does the agent decide when to clean and when to move?
3. What happens if both rooms are already clean at the start?
4. How would you modify the agent to handle **more than two rooms**?

Answer the Questions ⇩

1. **The agent follows two simple if-then rules: If the current room is dirty, the agent cleans the room. If it's clean, the agent moves to the other room.**
2. **The agent checks the condition of the room it is currently in, if the room is dirty, the agent cleans the room, if it's clean, the agent moves to the other room.**
3. **If both rooms are clean at the start, then the agent does not need to clean anything, since it will be moving to the next room, then back again to the other room since it is all clean.**
4. **To handle more than two rooms, simply adding more rooms using a list or dictionary should do the trick. The code is changed to implement this new list, and it can hold as much rooms as it can, while maintaining the if-then rules and number of steps.**